

1. SGLang 的 RadixAttention / RadixCache 解决什么问题？

RadixAttention 是 SGLang 后端的一项核心优化：在运行时自动复用共享前缀的 KV cache，从而减少重复的预填充（prefill）计算与显存占用，显著降低首 token 延迟并提升吞吐。

2. SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分？

page-sized KV cache：以固定大小的 page 为单位管理显存，维护逻辑页到物理页的映射，解决“KV 存哪里、怎么分配回收”。

prefix reuse:命中缓存的部分直接复用已有 KV，跳过对应 token 的 Prefill 计算，只对未匹配后缀做前向。解决“哪些计算可以跳过、复用哪段 KV”。

3.vLLM 的 PagedAttention 解决什么问题？它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异？

vLLM 的 PagedAttention 主要解决 KV Cache 的显存碎片和利用率低的问题，它借鉴操作系统虚拟内存分页，将 KV 切成固定大小的 block，按需分配并用 block table 映射逻辑块到物理块，使显存利用率提升到 90% 以上；SGLang 的 RadixAttention 与 paged KV 机制在内存分配层面与 PagedAttention 相通，同样以 page 为单位管理 KV，但进一步用基数树组织所有请求的 token 前缀，通过最长前缀匹配实现跨请求的 KV 复用，并支持 token 级 page_size=1 以及按匹配前缀长度排序的 cache-aware scheduling。两者的联系在于都采用分页式 KV 管理来消除碎片并支持前缀缓存，差异则在于 PagedAttention 以 block 哈希和 LRU 做前缀共享，更侧重分配效率，而 RadixAttention 以基数树和引用计数做更细粒度的前缀复用与调度优化，因此可以说 PagedAttention 是分配纪律，RadixAttention 是复用策略。